

CSS Animations

Bring Your Interfaces to Life

■ ANIMATIONS & TRANSITIONS

■ LEARNING OUTCOME

By the end of this module you will create smooth CSS transitions for hover effects, build complex keyframe animations, use transform properties for performant motion, and respect users' motion preferences.

■ Skills You Will Gain

- Apply CSS transitions with property, duration, timing-function
- Create keyframe animations with @keyframes
- Use transform: translate, rotate, scale, skew
- Understand which properties are GPU-accelerated
- Add animation-delay, iteration, direction, and fill-mode
- Respect prefers-reduced-motion for accessibility

■ Animation is the difference between a website that feels alive and one that feels dead. One well-timed transition can triple perceived quality and user engagement.

SECTION 1

CSS Transitions

What is a CSS Transition?

A transition animates the change between two states of a CSS property. Trigger: the change must happen — via :hover, :focus, class change via JavaScript, etc. Syntax: transition: property duration timing-function delay; property: WHAT to animate (or 'all' — avoid for performance) duration: HOW LONG (0.3s, 300ms) timing-function: SPEED CURVE (ease, linear, ease-in-out, cubic-bezier) delay: WAIT before starting

```
/* ===== BASIC TRANSITIONS ===== */ .btn { background: #E44D26; color: white; padding: 12px 24px; border-radius: 6px; border: none; /* Transition specific properties (not 'all' — performance) */ transition: background-color 0.2s ease, transform 0.15s ease, box-shadow 0.2s ease; } .btn:hover { background: #c43e1c; transform: translateY(-2px); /* lift up 2px */ box-shadow: 0 8px 24px rgba(0,0,0,.2); } .btn:active { transform: translateY(0) scale(0.97); /* press down */ box-shadow: 0 2px 8px rgba(0,0,0,.15); } /* ===== TIMING FUNCTIONS ===== */ /* ease: slow start, fast middle, slow end (DEFAULT) */ /* ease-in: slow start, fast end */ /* ease-out: fast start, slow end — BEST for most UI animations */ /* ease-in-out: slow start and end, fast middle */ /* linear: constant speed — use only for spinners/loops */ /* cubic-bezier(x1, y1, x2, y2): custom curve — create at cubic-bezier.com */ .smooth { transition: transform 0.3s cubic-bezier(0.4, 0, 0.2, 1); } /* Material Design */ /* ===== CARD HOVER EFFECT ===== */ .card { transition: transform 0.3s ease-out, box-shadow 0.3s ease-out; } .card:hover { transform: translateY(-8px) scale(1.02); box-shadow: 0 20px 40px rgba(0, 0, 0, 0.15); } /* ===== IMAGE OVERLAY EFFECT ===== */ .image-wrapper { position: relative; overflow: hidden; } .image-wrapper img { transition: transform 0.4s ease-out; } .image-wrapper .overlay { position: absolute; inset: 0; background: rgba(228, 77, 38, 0.8); opacity: 0; transition: opacity 0.3s ease; display: flex; align-items: center; justify-content: center; } .image-wrapper:hover img { transform: scale(1.1); } .image-wrapper:hover .overlay { opacity: 1; }
```

SECTION 2

CSS Transform

Transform Function	Effect	Example Value
translate(x, y)	Move element without affecting layout flow	translate(20px, -10px)
translateX(n)	Move horizontally	translateX(100%)
translateY(n)	Move vertically	translateY(-50%)
scale(x, y)	Scale size — 1=100%, 0.5=50%, 2=200%	scale(1.05) or scale(0.8)
rotate(angle)	Rotate clockwise (positive) or counter-clockwise	rotate(45deg) or rotate(-90deg)
skew(x, y)	Skew (distort) along axes	skew(10deg, 5deg)
matrix(a,b,c,d,e,f)	Combine all transforms in one matrix	matrix(1,0,0,1,20,30)

```
/* ===== GPU-ACCELERATED TRANSFORMS ===== */ /* These properties trigger GPU compositing — smooth 60fps animations */ /* SAFE to animate: transform, opacity */ /* AVOID animating: width, height, top, left, margin, padding */ /* (these cause layout reflow — janky, expensive) */ /* Sliding navigation */ .mobile-nav { transform: translateX(-100%); /* hidden off-screen */ transition:
```

```
transform 0.3s ease-out; will-change: transform; /* hint to browser – pre-promote to GPU */ }
.mobile-nav.open { transform: translateX(0); /* slide in */ } /* Rotation animation on button */
.loading-btn .spinner { transition: transform 0.3s ease; } .loading-btn:hover .spinner { transform:
rotate(180deg); } /* Multiple transforms at once */ .fancy-card:hover { transform: translateY(-8px)
scale(1.02) rotate(-1deg); } /* Transform origin – default: center */ .rotate-corner {
transform-origin: top left; transform: rotate(45deg); /* rotates from top-left corner */ }
```

SECTION 3

Keyframe Animations with @keyframes

```
/* Define the animation */ @keyframes fadeInUp { from { opacity: 0; transform: translateY(30px); }
to { opacity: 1; transform: translateY(0); } } @keyframes pulse { 0% { transform: scale(1); } 50% {
transform: scale(1.05); } 100% { transform: scale(1); } } @keyframes spin { from { transform:
rotate(0deg); } to { transform: rotate(360deg); } } @keyframes shimmer { 0% { background-position:
-200% center; } 100% { background-position: 200% center; } } /* Apply animations */ .hero-text {
animation: fadeInUp /* @keyframes name */ 0.6s /* duration */ ease-out /* timing-function */ 0.2s
/* delay before starting */ 1 /* iteration count (or 'infinite') */ normal /* direction: normal |
reverse | alternate */ forwards; /* fill-mode: keep final state after animation */ } /* Loading
spinner */ .spinner { width: 40px; height: 40px; border: 4px solid #e5e7eb; border-top-color:
#E44D26; border-radius: 50%; animation: spin 0.8s linear infinite; } /* Skeleton loading shimmer */
.skeleton { background: linear-gradient(90deg, #f3f4f6 25%, #e5e7eb 50%, #f3f4f6 75%);
background-size: 200%; animation: shimmer 1.5s infinite; } /* Staggered entrance – delay each item
*/ .item:nth-child(1) { animation-delay: 0.0s; } .item:nth-child(2) { animation-delay: 0.1s; }
.item:nth-child(3) { animation-delay: 0.2s; } .item:nth-child(4) { animation-delay: 0.3s; }
```

SECTION 4

Accessibility — Reduced Motion

```
/* Some users experience motion sickness from animations. Respect their OS preference with
prefers-reduced-motion. */ /* All animations off for users who requested reduced motion */ @media
(prefers-reduced-motion: reduce) { *, *::before, *::after { animation-duration: 0.01ms !important;
animation-iteration-count: 1 !important; transition-duration: 0.01ms !important; scroll-behavior:
auto !important; } } /* Better approach: start with no animation, add for users who are OK with it
*/ .fade-in { opacity: 1; transform: none; } @media (prefers-reduced-motion: no-preference) {
.fade-in { opacity: 0; transform: translateY(20px); animation: fadeInUp 0.5s ease-out forwards; } }
/* For spinners – substitute with opacity pulse (less vestibular impact) */ @media
(prefers-reduced-motion: reduce) { .spinner { animation: none; opacity: 0.7; } }
```

■ PRACTICE Q & A

Q1. What is the difference between a CSS transition and a CSS animation?

A: *Transition: animates change between TWO states triggered by an event (hover, focus, class change). Simple, no @keyframes needed. Animation: plays automatically with @keyframes, can have multiple steps, run infinitely, and play without user interaction.*

Q2. Why should you animate transform and opacity instead of left/top/width?

A: *transform and opacity are GPU-accelerated — they run on the compositor thread without triggering layout recalculation. Animating left, top, width, or height triggers expensive layout reflow on every frame, causing janky animations.*

Q3. What does animation-fill-mode: forwards do?

A: *The element keeps the CSS values from the last keyframe after the animation finishes. Without it (default: none), the element snaps back to its original state when the animation ends.*

Q4. What is will-change and when should you use it?

A: *A hint to the browser to pre-optimize an element for GPU compositing before the animation starts. Use sparingly on elements that WILL animate — e.g. will-change: transform on a sliding menu. Overusing it wastes memory.*

Q5. Why should you always include a prefers-reduced-motion media query?

A: *Some users experience nausea, dizziness, or seizures from motion/animation due to vestibular disorders. Respecting their OS preference (Reduce Motion setting) is both a legal accessibility requirement and basic good design.*

■ CSS Animations Cheat Sheet

<code>transition: prop dur ease</code>	Animate property change
<code>transition: all 0.3s ease</code>	Animate all changes (avoid)
<code>ease-out</code>	Best for UI – fast then slow
<code>transform: translateY(-8px)</code>	Move up – lift effect
<code>transform: scale(1.05)</code>	Grow slightly – hover effect
<code>transform: rotate(45deg)</code>	Rotate clockwise
<code>@keyframes name {}</code>	Define animation steps
<code>animation: name dur ease</code>	Apply keyframe animation
<code>animation-iteration-count: infinite</code>	Loop forever
<code>animation-fill-mode: forwards</code>	Keep final state
<code>will-change: transform</code>	GPU acceleration hint
<code>prefers-reduced-motion: reduce</code>	Respect motion settings